Module 10) Java – Hibernate Framework

1. Introduction to Hibernate Architecture

Que 1: What is Hibernate?

- Hibernate is a Java ORM (Object Relational Mapping) framework that maps Java objects to database tables.
- It helps perform database operations without writing much SQL.

Que 2: Definition and purpose of Hibernate as an ORM (Object Relational Mapping) tool.

- **Definition of Hibernate**
  - Hibernate is a Java-based ORM (Object Relational Mapping) framework that maps Java objects to database tables and allows developers to interact with the database without writing SQL.
- **Purpose of Hibernate**
  - To simplify database operations by converting Java objects into database records automatically
  - To reduce the need for writing complex SQL queries
  - To provide database independence and easy data management (CRUD operations)

Que 3: Comparison between Hibernate and JDBC.

Hibernate	JDBC
Object-based (ORM)	SQL-based
Less SQL code	More SQL code
Database independent	Database dependent
Automatic mapping	Manual mapping

Que 4: Why use Hibernate? (Advantages)

- Database independent
- Automatic table creation
- Less SQL code
- Supports HQL (Hibernate Query Language)
- Easy CRUD operations
- Better performance (caching)

Que 5: Hibernate Architecture (Components)

1. **Session Factory**
   - Loads configuration and creates sessions
2. **Session**
   - Main interface between Java application and database
3. **Transaction**
   - Manages database operations (commit/rollback)
4. **Query**
   - Used to execute HQL queries
5. **Criteria**
   - Used to build dynamic queries

Que 6: How Hibernate Works (Internal Flow)

1. **Load Configuration**
   - Hibernate reads configuration file (hibernate.cfg.xml) and mapping details
2. **Create Session Factory**
   - Session Factory is created using configuration (one per application)
3. **Open Session**
   - Session is opened to connect Java application with database
4. **Begin Transaction**
   - Transaction starts for performing operations safely
5. **Write & Execute Query**
   - Use HQL / Criteria / SQL to perform CRUD operations
6. **Hibernate Converts to SQL**
   - HQL is internally converted into SQL and sent to database
7. **Database Execution**
   - Database executes query and returns result
8. **Commit Transaction**
   - Changes are saved permanently (or rollback if error)
9. **Close Session**
   - Session is closed after work is done

2. Hibernate Relationships (One-to-One, One-to-Many, Many-to-One, Many-to- Many)

Que 1: Object Relationships in Hibernate

- Hibernate manages relationships by mapping Java objects (classes) to database tables using annotations.
- It automatically handles foreign keys and joins between tables.

Que 2: Types of Relationships

1. **One-to-One**
   - One object is related to one object
   - **Example:** One Person → One Passport
2. **One-to-Many**
   - One object has multiple related objects
   - **Example:** One Author → Many Books
3. **Many-to-One**
   - Many objects are related to one object
   - **Example:** Many Students → One College
4. **Many-to-Many**
   - Many objects are related to many objects
   - **Example:** Many Students ↔ Many Courses

Que 3: Mapping Relationships in Hibernate

- Hibernate uses annotations to define relationships:

  @OneToOne → One-to-One mapping

  @OneToMany → One-to-Many mapping

  @ManyToOne → Many-to-One mapping

  @ManyToMany → Many-to-Many mapping

- These annotations are placed in entity classes.

Que 4: Owning Side & Inverse Side

- **Owning Side** - Controls the relationship (contains foreign key)
- **Inverse Side** - Mapped by owning side using mappedBy
- Only owning side updates the relationship in database.

Que: 5 Cascade Types in Hibernate

- **Definition:**
  - Cascade = automatic operation on related entities

- **Common Cascade Types**

  1. **CascadeType.ALL**
  - All operations (save, update, delete, etc.) are applied to related entities

  2. **PERSIST**
  - When parent is saved, child entities are also saved

  3. **MERGE**
  - When parent is updated, child entities are also updated

  4. **REMOVE**
  - When parent is deleted, child entities are also deleted

  5. **REFRESH**
  - Refreshes child entities when parent is refreshed

  6. **DETACH**
  - Detaches child entities from session when parent is detached

- **How Cascade Affects Related Entities**
  - Operations on parent entity automatically affect child entities
  - Reduces manual coding for handling related data
  - Helps maintain data consistency

- **Example (Simple):**
  If Author → Books has CascadeType.ALL
  ✓ Saving Author → Books also saved
  ✓ Deleting Author → Books also deleted

---

3. Hibernate CRUD

Que 1: CRUD Operations in Hibernate

1. **Create (Insert)**
   - Used to add new records into the database
   - Use session.save(object)
2. **Read (Select)**
   - Used to fetch data from database
   - Use session.get() or HQL query
3. **Update**
   - Used to modify existing records
   - Use session.update(object)
4. **Delete**
   - Used to remove records from database
   - Use session.delete(object)

Que 2: What is HQL?

- HQL is an object-oriented query language used in Hibernate to interact with the database using Java objects instead of table names.

Que 3: Basics of HQL and how it differs from SQL.

HQL	SQL
Works on objects	Works on tables
Database independent	Database dependent
Uses class names & properties	Uses table & column names

Que 4: CRUD using HQL

- **Insert -** Usually done using save() (HQL insert is rarely used)
- **Select -** from Student
- **Update -** update Student set name='A' where id=1
- **Delete -** delete from Student where id=1

Que 5: Criteria API (Dynamic Queries)

- Used to create queries dynamically (without writing HQL)
- Based on Java objects and conditions
- **Example use:**
  - Filtering data
  - Adding conditions at runtime
